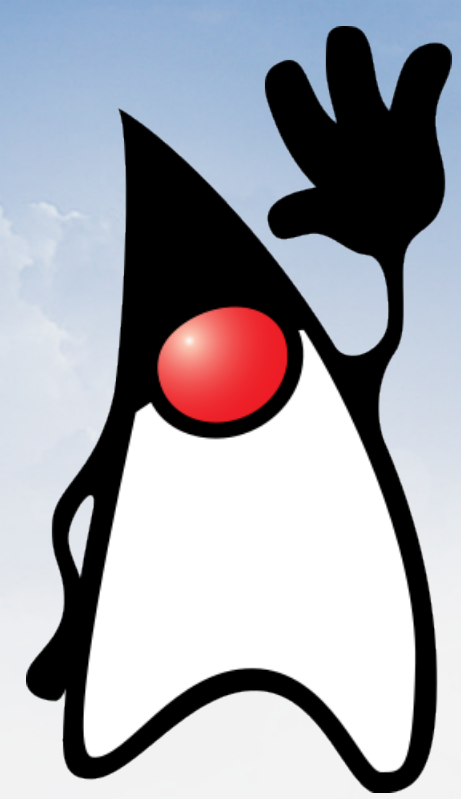




Methods and Global Variables

COP 3330
Department of Computer Science
University of Central Florida
Dr. Bacanli

Always initialize inside methods!

```
public class HelloWorld {  
    public static void main(String[] args) {  
        int a;  
        System.out.println(a);  
    }  
}
```

error: variable a might not have been initialized
System.out.println(a);

Global variable

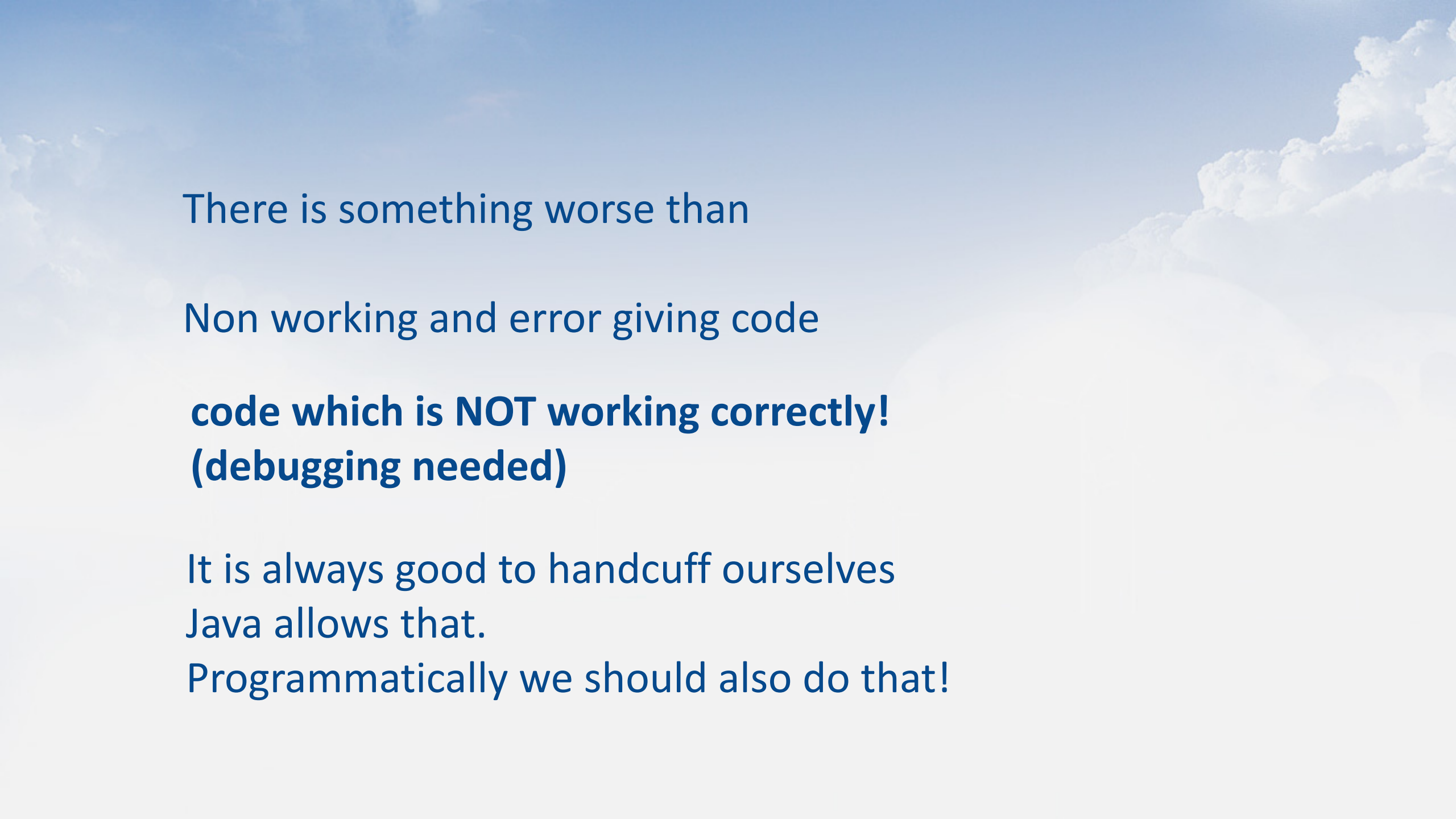
- How to use the global variable?

```
package HelloWorld;
public class HelloWorld {
    static int value; // default to 0

    public static void main(String[] args) {
        value = 7; //modified
        System.out.println("Value is "+value);
    }
}
```

Global variables defined outside of the functions can be accessed/modified by all functions.

Value is 7



There is something worse than

Non working and error giving code

code which is NOT working correctly!
(debugging needed)

It is always good to handcuff ourselves
Java allows that.

Programmatically we should also do that!

final

- final is for defining constant!
- As an informal rule define it in capitals!
- While reviewing, try to get compilation error as much as possible!

```
package HelloWorld;
public class HelloWorld {
    static final int VALUE=77;
    public static void main(String[] args) {
        VALUE=7;
    }
}
```

Once set, it can not be changed!

error: cannot assign a value to final variable VALUE

final

- Can we set the final value later?

```
package HelloWorld;
class HelloWorld {
    static final int VALUE;
    public static void main(String[] args) {
        VALUE = 7;
    }
}
```

If not set when created,
it can not be set later.

error: cannot assign a value to final variable VALUE

```
package HelloWorld;
// This class shows the scoping example and global variable concept
public class Test{
    int val=6;
    public static void main(String[] args){
        val=9;
        System.out.println(val+ " is inside the val as a global variable");
    } //end of main method
}
```

error: non-static variable val cannot be referenced from a static context

static methods can only access static things (methods and variables)
since main is static as a rule, every other stuff should be static also.

static int val = 6; //is needed

Variables defined outside will be global. Can be accessed by any function

Methods in Java

- Java uses *methods* term instead of functions in C
- Until we see Object oriented programming all the methods are *public static*.

Parameters



```
public static float add(int x, float y){  
    return x+y;  
}
```

```
//When calling add method  
add(3, 4.0f) ;
```

Arguments



Methods in Java

```
public static void printResult(float res){ //expects a float
    System.out.println("Division result "+result);
}
```

```
//This function is for adding integers
// preconditions: y is not zero!
//post condition of the function returns the division as integer
public static int divide(int x, int y){
    int resultDivision = x/y;
    return resultDivision;
}
```

```
public static void main(String[] args){
    int result = divide(6,2);// divide gives
    float convVal = result; //conversion
    printResult(convVal);
}
```

Trick (example of void method)

```
// Author Safa
// NID: ab1234
// This class salutes class
package firstJavaProgram;
public class FirstJavaProgram{

    public static void p(String x){
        System.out.println(x);
    }
    public static void main(String[] args){
        p("Hi Class!");
        int age=27;
        p(age+"");//printing an int
    }
}
```

Does order of function implementation matter?

No

Method Using Global variable

```
package CounterCode;
public class CounterCode {
    static int count = 7;

    public static void increaseCounter(){
        count++;
    }

    public static void main(String[] args) {
        System.out.println("Counter is "+count);
        increaseCounter();
        System.out.println("Now it is "+count);
    }
}
```

IncreaseCounter()
updates, main method
reads the value.

Value is 7
Now it is 8

Java methods

Java is pass by value

- Meaning the copies of the arguments will be given to the functions
- There is no user manipulatable reference in Java.(& does not exist)

For primitive types, we can say the copy of the value is passed.

```
package examplePack;

public class test{

    public static void main(String[] args) {
        int a=5;
        int b=3;
        pr(a+" "+b);
        swap(a,b);
        pr(a+" "+b);
    }

    public static void pr(String x){
        System.out.println(x);
    }

    public static void swap(int x, int y){
        int temp=x;
        x=y;
        y=temp;
    }

}
```

VERY IMPORTANT

What does this code do?

This is an example of a wrapper function. A function taking some parameters and mostly doing one thing with adjusted parameter/syntax.

This function operates on copy of the arguments. Copy of 5 and copy of 3 is given. The copies are swapped then they are purged from memory. Originals stay in the memory.

Algorithm design principle

- When we try to solve a problem we divide it into subproblems
 - Subproblems can be implemented as a method.
 - Multiple method will work together to solve a problem.
- We stick with the idiom “Divide and conquer”
- Two approaches
 - Top Down structured design
 - Bottom up structured design
- Personally I follow hybrid sometimes.

Top down structured design

void printPluses(int linenumber) function: Assuming linenumber = 5

start counter from 1

while counter <= linenumber

print *counter* number of plusses

go to the next line

increase counter by 1

//precondition: nothing

//postcondition: do not go to next line

printPluses(numberOfPlusses) function :

start counter from 0

while counter < numberOfPlusses

print + sign, dont go to next line

increase counter by 1

let's write a function to print this pyramid
if number of lines are given as parameter!

```
+  
++  
+++  
++++  
+++++
```

We can think this problem as
We will visit each line, one after another.
In each line visit,
we will call a method to print plusses.

```
package pyramidPackage;

public class Pyramid{
    public static void main(String[] args) {
        printPyramid(5);
    }

    // this function prints the pluses, does not pass to next line
    public static void printPluses(int numberOfPluses){
        for(int j=0; j<numberOfPluses; j++){
            System.out.print("+");
        }
    }

    //this function prints a pyramid with pluses
    public static void printPyramid(int linenumber){
        for(int j = 1; j <= linenumber; j++){
            printPluses(j);
            System.out.println(); // means just go to next line
        }
    }
}
```

methods can call each other as long as they are static.

main method calls a static method. That static method calls another static method.

The index numbers start from 1 in the for loop because we are printing 1 plus at the first line.

non void functions and scopes

```
package testFunctions;
public class testFunc{
    static int num = 99;
    public static void pline(int cVar){
        num = constantFunc(cVar); //appropriate
    }

    public static int constantFunc(int cVar) {
        System.out.println("global num is "+num);
        long val = Math.round(Math.sqrt(cVar));
        return (int)val;
    }

    public static void main(String[] args) {
        int cVar=5;
        pline(cVar);
        constantFunc(cVar);// syntax-wise correct
    }
}
```

What are the problems in this code?

the code is correct however ethically there are problems. Let's write it.

variables live and die between { }

If the cVars are not the same variable in the outer scope use a different variable name.

nonvoid methods can be called like void, syntactically correct but ethically wrong, it is confusing

Calling non void function like void means we are not interested in the returned value of the function but we are interested in the other operations the function is doing.

Ethically correct code

```
/* Returns the number of quizzes in a course given in parameter as
string
* @param    name the name of the course
* @return    the number of sections in the course */
public int numOfSections(String name){
    int sectionNo = 0;
    if(name.equals("COP3330")){
        sectionNo = 4;
    }else{
        sectionNo = 3;
    }
    return sectionNo;
}
```

This syntax is for Javadoc. You may face some clarifications to write such comments.

Void functions

```
static int globalVar = 6;
public void foo(int number){
    if (number == 5){
        System.out.println("COP3330");
    }else{
        System.out.println(globalVar +1);
        return;
    }
    System.out.println("Done");
}
```

Void functions do not return any value. They make operations on the given parameters or global variables.

Ethically do not use *return* in the middle of the function. Use only one return statement for non void functions. For voids if possible do not use at all.

What does return do in main method?

It will stop the program

```
System.exit(0);
```

stops the JVM. DO NOT USE IT! in this course

Method binding

```
package methodTest;
public class Main {
    public static void method1(int x){
        System.out.println("int x "+x);
    }

    public static void method1(float x){
        System.out.println("float x "+x);
    }

    public static void method1(double x){
        System.out.println("double x "+x);
    }

    public static void main(String[] args) {
        double var = 123.4;
        method1(var);
        method1(123.4); //123.4 is double
        method1(123.4f); //123.4f is float
    }
}
```

As long as the parameters are different, you can create multiple methods with the same name.

- Please try what happens when you remove one of the method1s?
- Try to call int param version of method1

```
double x 123.4
double x 123.4
float x 123.4
```

Tricky Questions

- 1) Can a method return a method in Java?
- 2) Can we write implement method inside a method in Java?
- 3) Can a method return 2 values in Java?
- 4) Can we swap parameters inside a function in Java?
- 5) Can we change parameter of a function in Java for the outer scope?
- 6) Can static methods access non static variables?
- 7) Can we call *nonstatic* functions from main?
- 8) Can we access/read/modify *nonstatic* outside variables from main?